Causal-Validity Auditing for Learned Grasp-Candidate Scoring, with a Case Study in Cross-Embodiment Grasping

***Abstract*—A learned scorer that reranks grasp candidates before execution is valid only if every input feature is computable *before* the candidate is executed – a constraint we call causal validity, easy to violate silently since an offline evaluation looks identical whether a feature is valid or not. We formalize a pre-execution admissibility criterion and build a manual audit tool plus an automated static dataflow analyzer that infers feature provenance from source code. Applied to our cross-embodiment grasp-reranking pipeline (SO-ARM101 to Piper, in simulation), the criterion shows its necessity: a pooling effect replicated five times at $p<0.0001$ collapses to an exact null ($p=1.0000$) once invalid features are removed, and the automated analyzer catches a contamination bug the manual audit missed. We validate the criterion against two published systems, and report a second case study: a silent unit-scaling bug in a robot controller, fixed with measured gains (0% → 45–65% on two near-zero object categories). We also report a tested ledger of twelve ruled-out mechanisms spanning candidate generation and execution reliability, and a hardware-deployment architecture with a safety checklist. Causal-validity auditing is not a theoretical nicety: a team looking for this failure mode still misses it until the check is made mechanical.**

***Index Terms*—Causal validity, grasp candidate scoring, evaluation methodology, cross-embodiment transfer, static program analysis, robot grasping, learned reranking, negative results**

I. Introduction

Grasp-candidate reranking – training a lightweight scorer to select the most promising pose from a pool of candidates before executing any of them – is an established pattern in robotic manipulation [1], [2]. Its correctness rests on an assumption rarely stated explicitly: every feature the scorer consumes must be computable *before* the chosen candidate is executed. A feature that depends on execution's outcome – contact force during approach, positional drift during descent, a kinematic residual re-solved after the arm has already moved – cannot be known at the moment a not-yet-executed candidate must be scored, however predictive it looks offline.

This constraint is easy to satisfy by construction and easy to violate by accident. GraspGen [2] satisfies it by construction: its discriminator takes only a point-cloud encoding and a candidate pose at inference time, while training labels come from simulated physics outcomes. It is violated by accident whenever a feature logged for training-data purposes gets reused, unexamined, as a live-selection input – and the violation is invisible under the evaluation method such systems reach for first: offline accuracy on historical logs, which already contain every candidate's outcome and so cannot distinguish a valid feature from an invalid one.

We hit this failure mode directly while extending a validated single-embodiment grasp reranker (LGGSN [13], pairwise-BPR-trained on a 5-DoF SO-ARM101 arm) to a second, independently implemented 6-DoF platform (a Piper arm, MuJoCo/RoboSuite simulation). A cross-embodiment pooling effect replicated across five independent pilots at $p<0.0001$, effect size growing from +0.21 to +0.35 AUC as feature sets were enriched. Designing the live-execution test this result was building toward required, for the first time, identifying which features would actually be available to a not-yet-executed candidate. None of the features driving the five-times-replicated result qualified. Restricting to the causally valid subset collapsed the effect to an exact null ($p=1.0000$).

This paper is organized around that discovery, not around a single proposed algorithm for improving grasp success. Contributions:

1) A formal **pre-execution admissibility criterion** (section IV), including a subtlety required in practice: a feature may depend on sibling, not-yet-executed pool candidates without being causally invalid – conflating "depends on multiple candidates" with "depends on execution" is itself a documented failure mode of our own reference implementation.
2) A **reference audit tool**, enforced as an automatic training-time gate, externally validated against two recent published systems (sections IV-B and IV-C).
3) An **automated dataflow analyzer** inferring feature provenance from source code rather than manual annotation, which independently caught a contamination bug our manual audit missed and had reported as clean – with a measurable downstream consequence – and an **interprocedural extension** that resolves a function parameter's own provenance by tracing its real call sites, validated on a second, independently implemented codebase (section IV-E).
4) A **second, independent case study** in the same discipline: a silent double-scaling bug in a widely used composite robot controller, validated on three previously near-zero-success object categories (section V).
5) An **honest, statistically rigorous ledger of twelve ruled-out mechanisms** spanning candidate generation and the execution-time reliability problem our diagnostics converged on, none presented as solved (section VII).

6) A **hardware deployment architecture and safety checklist**, scoped plainly as not yet physically validated (section VI).

We do not claim to have solved cross-embodiment grasp transfer or the execution-precision problem our diagnostics repeatedly point at. We claim that trying to solve them surfaced a general, underappreciated evaluation failure mode with a formal characterization and a checkable fix – and that even a careful team applying that fix to its own work will still find real instances of it, which is worth reporting rather than hiding.

II. Related Work

Grasp candidate scoring. Dex-Net 2.0 [1] trains a grasp-quality CNN offline on millions of simulated grasp outcomes, then deploys the trained network – not a live simulation query – to rank real candidates from a depth image at inference time; execution-derived information appears only as training supervision. GraspGen [2] follows the same pattern with an on-generator-trained discriminator whose inference-time inputs are a point-cloud encoding and a candidate SE(3) pose. Neither work states the input/label distinction as a formal, checkable criterion; both satisfy it, which we use as external validation of our formalization (section IV-B) rather than as prior art for the criterion itself.

Path-aware grasp feasibility. Recent work on grasp-dependent feasibility prediction [6] generates causally admissible, path-level geometric labels – inverse-kinematics feasibility and a transit-collision flag from mesh sweeps along a fixed waypoint template – and trains a compact MLP on them for candidate ranking, validated as transferring to physics-executed trajectories on a single rigid cuboid with a fixed waypoint template. This is framed entirely as an efficiency argument (cheap labels vs. expensive physics simulation), with no connection to a causal-validity theory. We considered adapting this mechanism for our own still-open execution-reliability problem and rejected it (section VII) on the grounds that our diagnosed failure mode (sustained, necessary contact under sub-millimeter miscentering) does not match the binary avoidable-collision framing this mechanism targets, and is mechanistically closer to four contact-response interventions we had already tested and found ineffective.

Cross-embodiment transfer in grasping. GraspGen-X [4] demonstrates cross-embodiment 6-DoF diffusion-based grasping validated on a real AgileX Piper arm, using pointwise (not pairwise) supervision. Freeform Preference Learning [5] learns a language-conditioned reward model from trajectory-level human preferences for post-hoc reward shaping in reinforcement learning – a role that legitimately requires execution-derived input by design, which we use as a boundary case our criterion correctly does not flag (section IV-B).

Data leakage and offline evaluation. Look-ahead and label leakage are recognized general problems in machine learning. Concurrent work on offline policy evaluation for manipulation [7] studies a related but distinct integrity problem — truncation bias from finite-horizon episodes under classical TD/Monte Carlo evaluation, addressed via a discounted-liveness Bellman operator — rather than feature-level leakage; we cite it as adjacent evidence that offline manipulation-policy evaluation is an actively scrutinized area, not as prior art on our specific criterion. Closest to our formal criterion is concurrent work by Roth [9], a grammar of eight typed primitives with a call-time-enforced admissibility gate that makes train/test leakage structurally unrepresentable in standard tabular and time-series ML pipelines. Two differences are worth stating plainly. First, domain: Roth's boundary is the train/test split; ours is whether a candidate has physically executed yet, including a pool-relative case (a feature may depend on sibling, not-yet-executed candidates) with no analogue in a train/test framework. Second, mechanism: Roth's grammar requires authoring a pipeline against his primitives (reference implementations for Python and R); our tool statically analyzes arbitrary, already-written code via a single per-function marker, requiring no pipeline rewrite. We are not aware of prior work stating our specific criterion for grasp-candidate scoring features, nor of an automated tool for detecting the violation from unmodified source code in this domain.

Perception uncertainty for grasping. Closest to our planned real-noise-characterization extension (section VIII) is Consensus-Driven Uncertainty [10], which trains an MLP to predict grasp failure from disagreement among three independent pretrained pose estimators (EPOS, GDRNPP, ZebraPose) applied to the same real image, and reports that "poses perturbed by Gaussian noise will not fail the same way that real pose estimators fail" – independent evidence for the same hypothesis motivating our own noise-characterization plan. Two gaps remain open relative to our design: that work does not quantitatively characterize the noise's statistical structure (per-axis variance, Gaussianity, position-orientation correlation), and its uncertainty signal drives an accept/reject gate on a single proposed grasp rather than a selection rule over a candidate pool.

Contact-aware execution. CoorGrasp [8] and related work on wrench-balance-criterion phase transitions use real tactile sensing to coordinate arm motion after fingers have already formed contact with an object. Lacking tactile hardware, we used MuJoCo's own contact-force API as a privileged, hardware-free proxy for this class of mechanism (section VII), finding it – along with three related contact-response variants – ineffective for our specific failure mode.

III. System

Our platform is a 6-DoF AgileX Piper arm simulated in MuJoCo via RoboSuite, a second, independently implemented embodiment alongside this project's original 5-DoF SO-ARM101 platform. The core execution primitive drives the arm through a fixed phase sequence – transit, approach, descend, close, lift, transit-to-tray, lower, retract – using damped least-squares inverse kinematics solved against a saved kinematic snapshot, and an absolute joint-position

controller. Success is defined as final object position within a fixed radius of the tray center.

LGGSN, this project's validated single-embodiment contribution, is a lightweight MLP-based pairwise reranker trained with Bayesian Personalized Ranking loss over a 14-dimensional geometric feature space on the SO-ARM101 platform, and is the deployed, live candidate scorer for that platform's production pipeline. It is both reused infrastructure in this paper and, in section IV, the system under audit – the causal-validity criterion is applied directly to its live, already-deployed feature pipeline, not to a toy example.

IV. Causal-Validity Auditing for Learned Grasp-Candidate Scorers

A. Definitions

Definition 1 (Candidate pool, execution trace). A scene induces a candidate pool $C = (c_1, \ldots, c_N)$, each candidate specified by a pose $p_i \in SE(3)$ and generator-time attributes a_i. Let G denote static, execution-independent scene/object information. If c_i is physically executed, let T_i denote its execution trace – contact forces, intermediate poses, net displacement, and the terminal success label $y_i \in \{0, 1\}$.

Definition 2 (Selection-time information set). At the moment a scorer must choose which candidate(s) to execute, the selection-time information set is

$$I(C) = \{p_1, \ldots, p_N\} \cup \{a_1, \ldots, a_N\} \cup G \cup K(C, G),$$

where $K(C, G)$ is the set of values obtainable from isolated, execution-free queries against a candidate pose and G (e.g. a kinematic IK residual or collision check evaluated without moving the arm). $I(C)$ contains no T_j for any j – nothing in the pool has executed yet.

Definition 3 (PRE_EXECUTION-admissible feature). A feature function φ is admissible if there exists g such that, for every candidate c_i in every pool C, $\varphi(c_i) = g(I(C))$. $I(C)$ includes every candidate's pose, not just c_i's own – a feature may depend on sibling candidates (e.g. distance to the pool centroid) without being execution-derived. Conflating "depends on more than one candidate" with "depends on execution" is a documented failure mode of our own reference implementation (section IV-D). Otherwise φ is EXECUTION_DERIVED.

Proposition. A scorer $S(c_i) = h(\varphi_1(c_i), \ldots, \varphi_k(c_i))$ used to select among not-yet-executed candidates is causally valid iff every φ_j is admissible. A feature set containing an execution-derived φ_j is invalid regardless of how well S performs offline on historical (c_i, T_i) pairs – offline evaluation implicitly supplies T_i for every evaluated i, satisfying φ_j even though it would be undefined for a genuinely new candidate.

Corollary. Execution-derived quantities remain valid as supervision targets or as values used only to construct them – the constraint binds the input side of S, not the label side. This is why GraspGen's design (section II) is correct.

Algorithm. (1) Tag every raw field with provenance $\tau(f) \in \{\text{PRE}, \text{EXEC}\}$ by tracing the code path that computes it, not a comment or an assumption about a similarly named field. (2) Before training or invoking any live-selection scorer with feature set F, reject if any $f \in F$ has $\tau(f) = \text{EXEC}$ or is unregistered – fail closed. (3) Re-run (2) whenever F changes or whenever the code computing any $f \in F$ changes, since $\tau(f)$ is a property of code, not of a field's name.

B. External grounding

We checked the criterion against two recent published systems chosen to stress-test both directions of the boundary. GraspGen [2] is confirmed to satisfy it: its discriminator's inference-time inputs are a point-cloud encoding and candidate pose only (verified directly against the released code, commit `2dd8852`: `grasp_gen/models/discriminator.py`'s `forward` and its call site `grasp_gen/grasp_server.py::score_grasps_with_discriminator` whose only inputs are a centered point cloud and centered candidate poses); training labels are simulated shaking-motion outcomes. Freeform Preference Learning [5] is confirmed as a correct *non-flagged* boundary case: its reward model legitimately takes trajectory-level, execution-derived input, because its role in the pipeline – post-hoc reward shaping for reinforcement learning – is categorically different from live pre-execution candidate selection. The criterion correctly distinguishes "execution-derived input because the model's role requires it" from "execution-derived input leaked into a model whose role requires admissibility," which is the failure mode we hit.

C. Retrospective demonstration

We implemented the registry (field-level provenance tags with an enforced audit function) covering every raw field logged by both platforms' pipelines and ran it against every feature set used across this project's cross-embodiment reranking pilots, in chronological order. The audit correctly flags every pilot that produced the illusory pooling-beats-zero-shot effect and passes every pilot ultimately trusted; it attributes the contamination specifically to the Piper side (a descend-phase IK residual logged after physical motion, and a positional-drift measurement by definition only available post-motion), while the SO-ARM101 side's superficially similar fields – traced against the actual live inference code rather than a comment describing an inactive legacy dataset – turned out to be a harmless pre-execution proxy plus dead constant-zero features.

D. Self-corrections

Building this registry produced two errors of its own, both caught by re-tracing against live code rather than trusting an initial plausible explanation: (1) the SO-ARM101 fields above were initially mislabeled execution-derived by quoting a comment describing a different,

unused dataset; (2) two pool-relative features were initially mislabeled by conflating "depends on sibling not-yet-executed candidates" with "depends on execution." We report both because they demonstrate the discipline the method argues for, applied to itself, not because they weaken the tool's credibility.

E. Automated tagging

A manually populated registry does not scale and, as section IV-D shows, is itself error-prone. We built a static dataflow analyzer that infers provenance automatically: a fixed-point pass over the module identifies every *execution-touching* function (one that directly or transitively calls the simulator's physical-actuation entry point), and a forward taint-propagation pass over the target function's body – gated by a single human-placed marker denoting the point after which the current candidate has begun physically executing – determines, for every field in the function's output, whether it is reachable from any post-marker execution-touching call or live-state read. This reduces the manual annotation burden from O(number of logged fields) to O(number of physical commit points in the codebase) – a single marker, in our case, rather than thirteen-plus per-field judgments.

Run against the real, live pick-and-place function, the tagger independently disagreed with our hand-built registry on one field: it flagged a grasp-orientation feature as execution-derived, where the manual registry had marked it admissible. Tracing why confirmed the tagger was correct: the underlying pose variable is reassigned twice – once pre-commit during candidate selection (admissible) and once post-commit, at a deliberate mid-trial drift-correction step that re-reads simulator state after the descend phase has already executed. A human reading the function once sees the first assignment and does not track the second; the automated tagger mechanically follows every reassignment. This was not a cosmetic error: the field was part of the feature set our own prior retrospective validation (section IV-C, "corrected" configuration) had already reported as clean. Re-running that validation with a genuinely admissible two-feature set (dropping the contaminated field from both platforms for a fair shared feature space) confirmed the qualitative finding is unchanged – all pooling conditions remain exactly identical to zero-shot transfer (diff = 0.0000) – but the reported absolute accuracy changed from 0.8236 to 0.1327, a number that had already been published as final in this project's own working documentation before the automated check caught it.

We verified, rather than assumed, why: the object top-surface offset feature is an exact constant across every row of the single-object (Cracker-only) Piper collection (σ=0 over n=250), and the remaining pose-height feature is a near-constant *per scene* – 14/25 scenes show within-scene spread under 2×10^{-4}, consistent with floating-point/physics-settling noise rather than a genuine per-candidate signal, because it is read once, from the object's own spawn pose, before any candidate-specific action – identical regardless of which of the ten pooled candidates is later attempted. Together this means the corrected two-feature set has almost no capacity to discriminate between different candidates drawn from the *same* scene, which is exactly the comparison the pairwise ranking objective is trained and evaluated on. This explains the exact equality across all four pooling conditions (there is essentially nothing in the input for any of them to learn from) and reframes 0.1327 not as an unexplained oddity but as an expected consequence of a feature set that is causally valid yet structurally near-uninformative for this specific comparison – a genuinely predictive but still-admissible per-candidate feature (e.g. the original, pre-commit candidate orientation, distinct from the execution-derived value section IV-E identified) was not tested here and is left as future work.

We validated the tagger against a second, structurally different function (a pure data-serialization routine with no internal execution boundary and a constructor-call field pattern rather than a return literal) and initially found a genuine scope limitation worth stating rather than hiding: because this function contains no physical-commit marker of its own, every field trivially returns admissible, which happens to be correct (verified independently against the function's known input provenance) but was not actually *demonstrated* by the tool – the analysis was single-function-scoped and could not verify a parameter's own provenance across a call boundary.

We closed this by extending the tool with interprocedural resolution: for a target parameter, find every real call site in the module and evaluate the taint of the bound argument at that call site, using the calling function's own taint state up to that point, rather than assuming the parameter is safe. Two fixes were necessary on this second, independently implemented (PyBullet-based, not MuJoCo-based) codebase: the physical-actuation entry-point check, previously hardcoded to a single method name, had to be generalized to a configurable set, since this codebase's execution calls are named differently; and the calling function itself needed its own commit marker, since – consistent with the design already established – without one the tagger correctly flags nothing as tainted rather than guessing. This generalizes the earlier finding: interprocedural resolution does not shrink the marker requirement to one marker per codebase, it requires one marker per function with a genuine execution boundary, applied transitively up the call chain. With both fixes in place, resolving the real call site correctly and automatically recovered every field's true provenance – including confirming that a success-derived label field is execution-derived (correct, since it is the training label, exempt under the Corollary) while the pose-derived features are genuinely pre-execution and safe for live selection.

F. Controlled fixture validation

The internal case studies above show the tagger catching real bugs, but do not by themselves bound its detection

TABLE I
CONTROLLED FIXTURE VALIDATION, $n = 30$ LABELED FIELDS.

Metric	Value
Accuracy	0.933
Precision (EXECUTION_DERIVED positive)	0.889
Recall	0.889
F1	0.889
False positive rate	0.048

rate: a clean scan of unfamiliar code is uninterpretable without knowing how often the tool misses a true violation or flags a false one. We built a labeled benchmark of 14 fixture functions spanning 12 categories and 30 individually labeled fields (clean pre-execution assignment, direct execution-derivation, multi-hop taint propagation, post-commit reassignment, dead-constant fields behind a misleading comment, a pre-marker settle step reusing a live method name, field-name collision, aliased/renamed environment handles, static configuration reads, interprocedural taint through a helper, mixed-label baseline padding, and subscript-assignment field writes – the last category added after auditing real external code, section IV-G). Each fixture's expected label was fixed before running the tagger, and several categories were deliberately designed to characterize a known heuristic boundary rather than to be trivially passable.

Of 30 labeled fields, 28 were classified correctly; the two disagreements are both pre-registered, expected characterizations of the tool's static heuristics, not surprises found after the fact. **Aliasing false negative**: the tagger's execution-touching checks match a hardcoded variable name (`env`); a physical-actuation handle passed under any other name (a renamed parameter, `self.env`, `sim`) is invisible to it. **Static-configuration false positive**: the same heuristic that lets the tagger recognize live-state reads ("any attribute chain rooted at a variable named `env`") cannot distinguish a static, construction-time configuration read from a genuine live-state read – both are syntactically identical `env.*` access. Both are stated here as known limitations of the current static implementation, not smoothed into the headline numbers.

One methodological note is worth keeping rather than hiding: two of our own hypotheses about the aliasing case were wrong before the fixture suite was actually run. The original design intent assumed the tagger would miss an execution call under an *unrecognized method name* (e.g. `env.advance_physics(...)` against a fixed list of known entry-point names); two fixture attempts built around that mechanism both passed, because the actual check walks any attribute chain rooted at `env` regardless of the specific method name, making the assumed method-name list irrelevant to this code path. Only reading the exact AST condition – not the docstring describing it – revealed the real, narrower mechanism (the hardcoded variable name), and the fixture was rewritten a third time around that. The same discipline this tool exists to enforce on other people's code – read the code, not a comment describing it – turned out to be necessary to characterize the tool itself correctly.

G. External boundary checks

Section IV-B checked the criterion conceptually against two published systems. Here we report what happened when the automated tagger (section IV-E) was pointed at code we did not write, with no prior familiarity, and no marker placed until after independent verification. These are boundary/specificity checks against two systems, not a benchmark: we make no claim that either codebase is exhaustively verified clean, only what is reported for the specific paths inspected.

Auditing Sim-Grasp [3] (`grasp_simulation.py::main_simulation_loop`) exposed a field-writing pattern the tagger could not see at all: a result written via subscript assignment to an existing container (`new_candidates[obj]["grasp_samples"][idx]["simulation_qua = 1`), rather than the `return {...}` or `dict(k=v)` patterns the tool previously recognized. We extended the tagger to recognize this pattern (Category 12, section IV-F) and re-validated the full fixture suite afterward – identical accuracy, no regression on any previously-passing category. This is the value of external boundary checks distinct from a fixture benchmark: a fixture suite can only characterize failure modes someone already thought to construct, while unfamiliar real code surfaces coverage gaps a benchmark's author would not have anticipated.

Applying the extended tagger to the same function without a commit marker flags every field as PRE_EXECUTION by design (section IV-E's fail-closed convention: no marker means no execution boundary has been asserted, so nothing can be shown tainted) – correctly reported as not yet meaningful evidence on its own. Manually tracing the call graph before placing a marker showed why a marker would have been the wrong next step: `main_simulation_loop` runs a post-placement stress test (lowers a supporting surface under the object, checks whether it stayed in place) and writes `simulation_quality` as the resulting label – it is not a candidate-scoring function at all. The candidate pose is committed earlier, by `RobotSpawner.spawn` placing the gripper directly at the candidate's translation and rotation, before `main_simulation_loop` is ever called. Labels are explicitly exempt from the PRE_EXECUTION requirement under our own Corollary, so `simulation_quality` being execution-derived is correct and expected – treating this as a discovered violation would have been an unsupportable claim, caught by the same manual-verification discipline this method argues for, applied to itself, before any marker was placed or any claim written down. We report this not as a violation found in Sim-Grasp, but as the audit *process* correctly declining to make an unsupported claim – a legitimate, if less dramatic, external-validation outcome in its own right.

Taken together, sections IV-F and IV-G bound what this tool can be trusted to do: it is a static analyzer, and

pure static analysis cannot always establish the actual runtime path taken by dynamic Python (dynamic dispatch, monkey-patching, config-driven branching are all outside its reach). Every finding reported anywhere in this paper that came from the tagger was manually verified against the underlying code before being trusted, not accepted on the tool's output alone – the detection-rate numbers in table I describe how often that manual step would be needed to catch a miss, not a guarantee that it would always succeed at doing so on unfamiliar code. A hybrid static-plus-runtime-taint design would be more defensible than a static-only one; this is a stated scope limitation of the current tool, independent of how clean any particular scan turns out to be.

H. An independently discovered infrastructure bug, and a stronger re-verification

While instrumenting a new admissible feature to test directly (whether the original, pre-commit candidate orientation – distinct from the disqualified execution-derived value in section IV-E – carries real signal), we found a second, unrelated bug: the Piper simulation's object-placement sampler was constructed without an explicit random-number generator, silently falling back to an OS-entropy-seeded default completely independent of the `numpy` legacy seed this project's every prior experiment relied on for reproducibility. Concretely, a scene's ten pooled candidates were never actually facing the same object placement, despite the code's own documentation asserting they were; confirmed empirically by re-running an unmodified data-collection script on an identical scene twice and observing three different outcome patterns across three runs, and independently by finding 7–9 cm of spawn-position spread within already-collected data that should have been constant. We fixed this by explicitly deriving a seeded generator from the legacy random state at scene-construction time, verified by confirming bit-identical object placement across repeated runs of the same seed.

This bug does not affect sections IV-B to IV-E: the causal-validity criterion and every provenance verdict in this paper come from tracing code, not from statistical properties of collected data. It does affect any specific accuracy number computed on data grouped by an assumption of shared placement. We therefore re-collected the 25-scene Piper dataset under the fix and re-ran the corrected two-feature evaluation: the qualitative finding is unchanged – zero-shot, pooled-`none`, pooled-`additive`, and pooled-`interaction` remain exactly identical (0.1036, diff = 0.0000) – confirming the paper's core claim is not an artifact of the placement bug.

We then tested the untested admissible feature this section had flagged as future work. A naive pooled check looked dramatic: candidate orientation correlates with success at $r = -0.55$ ($p<0.0001$, $n=250$) – far stronger than any other signal in this investigation. Decomposing it, following this paper's own stated discipline rather than reporting the encouraging number, revealed why: mean orientation and success rate correlate strongly *between* scenes ($r = -0.71$, $p=0.0001$ across 25 scenes) because different object placements are both associated with systematically different orientations and, for unrelated geometric reasons, different difficulty – but the *within*-scene correlation, the only one that could actually inform a live selection decision among candidates facing the same already-placed object, is indistinguishable from zero (mean +0.03 across 10 mixed-label scenes, no individual scene reaching significance). A feature that appears, by a wide margin, to be the strongest predictor found in this entire investigation carries zero real selection-relevant signal once correctly decomposed – a sharper demonstration of this paper's thesis than the original finding it was built to test.

I. A complementary question: causal validity vs. accuracy relevance

Section IV-D validated `dist_to_centroid` and `z_rel` as PRE_EXECUTION-admissible; both ship in `full_v2`, LGGSN's live SO-ARM101 checkpoint (14-dim: the 12 fields of section III's baseline plus these two). Admissibility answers whether a feature is *safe* to use, not whether using it *helps* – a question we tested directly, holding admissibility fixed, using the same paired statistical methodology as section VII: four checkpoints trained identically except for this feature pair (`base`: neither; `nodist`: `z_rel` only, despite the name; `nozrel`: `dist_to_centroid` only; `full_v2`: both), evaluated pairwise on 582 real, provenance-pinned, identity-aligned validation pairs – exact two-sided McNemar's test on discordant pairs, Holm–Bonferroni-corrected jointly across all five comparisons, cluster-bootstrapped 95% CIs resampled by query since pairs sharing a query are not independent draws (`research_agent_pilots/lggsn_analysis/`).

`z_rel` alone significantly *hurt* ranking accuracy relative to neither feature (`base` 65.5% vs. `nodist` 52.6%, −12.9pp, $p<0.0001$ raw and Holm-adjusted). `dist_to_centroid` alone showed no significant effect (`base` 65.5% vs. `nozrel` 68.7%, +3.3pp, $p=0.099$ raw / $p=0.296$ Holm) – but a follow-up power analysis, computed from the same discordant-pair counts with no new data, found this comparison had only 39% post-hoc power at $\alpha=0.05$ and would need roughly 2.6× more discordant pairs to resolve at 80% power; we report it as inconclusive, not as evidence of no effect. Restoring `dist_to_centroid` alongside `z_rel` (`full_v2` vs. `nodist`) recovered the loss with a significant positive effect (+13.2pp, $p<0.0001$ raw and Holm) – nearly canceling `z_rel`'s isolated harm – while both features together against neither (`base` vs. `full_v2`) is a genuine near-null (+0.3pp): power analysis puts the discordant pairs required to resolve it at 80% power above 38,000, well beyond a realistic follow-up, which is itself a more informative reading than "underpowered." The fifth comparison, isolating `z_rel`'s marginal effect in the presence of `dist_to_centroid` (`full_v2` vs. `nozrel`), was also inconclusive (−2.9pp, $p=0.221$ raw / $p=0.442$ Holm, 25% post-hoc power).

We do not read this as evidence of a genuine feature interaction – each configuration is one trained checkpoint,

not a replicated ablation, so we cannot separate a real interaction effect from ordinary training-run variance. What the data do support, without that further claim: (1) `z_rel` measured in isolation is not accuracy-neutral, and shipping it without `dist_to_centroid` would have been the worse configuration of the four tested; (2) the currently deployed configuration (`full_v2`, both features) is statistically indistinguishable from the simplest one (`base`, neither) on this validation split, and – unlike the other two null comparisons here – this is not merely an underpowered study: the effect size itself is too small for any realistic amount of additional paired data to resolve. This complements, rather than substitutes for, section IV-D's admissibility finding: both features are safe to use; whether either is worth the added model complexity is a separate, now directly measured question, and, for the specific pair tested together, the answer on this split is "not detectably." Pairwise ranking accuracy on a fixed validation split is not a grasp success rate, and this ablation does not license a claim about physical grasp performance.

V. Case Study: A Silent Gripper-Controller Scaling Bug

While investigating why grasp success generalized poorly across object categories on the Piper platform, we traced the failure one level below the usual diagnostic stopping point – not "where does the arm end up," but "what does the gripper actually command at the actuator level" – and found that RoboSuite's composite gripper controller was silently re-scaling an already-real-units gripper action a second time. The platform-specific gripper's action-formatting function computes and returns an absolute, real-units joint-position target directly (a porting artifact from a different upstream convention), but the composite controller's default action-scaling behavior applies its own normalized-to-real-units rescaling on top of it, unconditionally. At a verified, fully converged "fully open" command, the actual commanded actuator range spanned roughly one tenth of a millimeter, against a documented 76–100 mm. A first fix attempt – supplying explicit input-range bounds – had no effect, traced to the controller's config-assembly code discarding those keys during rebuild; the actual fix was a single boolean flag, disabling action scaling for this sub-config, letting the already-correct action pass through unmodified.

A paired n=20-per-object pilot validated the fix: Cracker 0% → 45%, Mustard 0% → 65%, Pear unaffected at 65% (consistent with its prior 70–90% range within pilot-scale noise). A final confirmatory run at paper scale (n=40–60 per object, on a clean, previously unused trial range) reproduced Cracker (50%, n=40) and Mustard (70%, n=40) cleanly, but Pear did not replicate at its original figure – three independent 20-trial batches gave 40%, 40%, and 50% (pooled 43.3%, n=60), consistently below the original single-batch 65% (Fisher's exact p=0.12). No code drift was found that would explain a genuine regression; we report the larger, replicated figure (43.3%) as the confirmed rate rather than the flattering first number, consistent with this paper's stated discipline.

VI. Toward Real-Hardware Deployment

Following our project's scope decision to include real-hardware capability rather than remain simulation-only, we built an interface skeleton against the documented vendor SDK for our target 6-DoF arm, written while the physical arm was not connected to the development machine. Every hardware-touching method raises an explicit error with a verification marker rather than guessing an unconfirmed unit or scale convention, following the same relative-delta-clamped, replay-not-live-IK design pattern used by this project's validated SO-ARM101 hardware track. We present this as an architecture and an itemized pre-deployment safety checklist, not as a hardware result – physical validation is explicitly future work, gated on the checklist items stated in the design document.

VII. What Doesn't Work: An Honest Negative-Results Ledger

Table II summarizes twelve mechanisms tested and closed over the course of this investigation, each evaluated with a paired-trial design and McNemar's exact test (or the noted equivalent). Full mechanistic explanations for each row are given in appendix A.

*Row 4 and rows 7–10 were re-run after section IV-H's placement-RNG fix; see below – the consensus-selection null (row 4) was reconfirmed under corrected pairing, and three of the four rows 7–10 negative findings did not replicate.

Rows 7–10 were originally reported as four independently negative mechanisms. Re-verifying them under section IV-H's placement-RNG fix told a materially different, more precise story: **three of the four (hard stop, sustained-contact stop, admittance damping) have no measurable effect under valid pairing** – their original "made things worse" findings were very likely themselves artifacts of the same RNG bug, which added spurious between-condition variance that looked like a real effect (row 9's re-verification is especially stark: 75% vs. 75% with *zero* discordant pairs across n=20, meaning the binary outcome matched on literally every trial). **Only row 10 (pre-narrowing gripper clearance before descent) has a genuine, replicated effect**, and it strengthened substantially under correction (p=0.0005 vs. the original p=0.11, not significant). The corrected reading: contact-*response* interventions (react once contact during descent is detected) are inert for this failure mode, most likely because they rarely trigger in a way that matters; the one intervention that changes baseline geometry *before* any contact occurs is the one with a real cost. This is narrower and more mechanistically precise than "everything we tried backfired," and it is the reading we stand behind. We state plainly that the underlying failure mode is diagnosed, not solved, and requires either physical tactile sensing or a mechanism fundamentally different from the four tested here.

TABLE II
RULED-OUT MECHANISMS (SEE APPENDIX A FOR FULL DETAIL).

#	Mechanism	Result
1	OT-coupled flow matching for candidates	−10.0pp vs. baseline (p=0.0025); negative on all 7 objects
2	Condition-aware OT fix (C^2OT)	Partial, inconsistent mitigation
3	Real-time world-model action correction	Net negative, 3 independent pilots (−9.3, −18.7, −13.3pp)
4	Training-free consensus candidate selection	Effective on SO-ARM101; did not replicate cross-embodiment (re-verified*, null confirmed)
5	AR-mediated active-calibration sampling	Active worse than passive (p=0.0048); refinement null
6	Cross-embodiment reranking (causally invalid)	$p<0.0001\times5 \rightarrow$ exact null (p=1.0000) once corrected – section IV
7	Binary contact-triggered hard stop	Re-verified*: null (65% vs. 65%, p=1.0000)
8	Sustained-contact stop (5-step)	Re-verified*: null both batches (p=1.0000)
9	Force-proportional admittance damping	Re-verified*: null, 0 discordant pairs
10	Gripper pre-narrowing before descent	Re-verified*: worse, 75% vs. 15%, **p=0.0005**
11	World-model sim-to-real, scarce real data	Not piloted – closed at feasibility stage
12	Energy-based candidate scoring (EBM)	Naive version catastrophic (0–16% across 3 objects); hard-negative-mined fix reaches baseline parity (pooled −3.7pp, p=0.29; significant win on one object, p=0.041)

VIII. Conclusion

We set out to improve cross-embodiment grasp reliability and found, instead, that our own strongest apparent result in that direction was an artifact of a silent, well-known-in-the-abstract but rarely formalized class of evaluation error. Rather than treat that as a dead end, we formalized the constraint it violated, built and validated both a manual and an automated tool to check for it, and found – twice – that even careful, motivated verification still misses real instances of the problem it is looking for, until the process is made mechanical. Alongside this, we report a genuine, validated fix to a previously undiagnosed execution bug that unlocked most of a robot platform's latent grasp reliability, an honest account of twelve mechanisms that did not work for the problem that remains, and an architecture for the real-hardware phase this project has not yet reached. We believe the discipline demonstrated across all of these – verify against ground truth, report the number that survives replication rather than the one that looked best first, and say plainly what remains unsolved – is a more durable contribution than any single mechanism we tested.

As a next step, motivated by independent evidence [10] that synthetic Gaussian perturbation does not reproduce real pose-estimator failure modes, we are instrumenting a real RGB-D sensor to empirically characterize pose-estimation noise structure (per-axis variance, Gaussianity, position-orientation correlation) against the isotropic-Gaussian assumption our own candidate-selection experiments relied on, and – contingent on that measurement actually showing a structural difference, not assumed in advance – to test whether a selection rule informed by the measured structure changes the cross-embodiment generalization result in table II row 4. Any such feature remains checkable against section IV's criterion, since it would be computed purely from the candidate pool's own poses and the measured noise covariance. This work has not yet started data collection and is not included in the results reported above.

Appendix

Every entry was tested with a paired-trial design and McNemar's exact test (or the noted equivalent).

References

[1] J. Mahler, J. Liang, S. Niyaz, M. Laskey, R. Doan, X. Liu, J. A. Ojea, and K. Goldberg, "Dex-Net 2.0: Deep Learning to Plan Robust Grasps with Synthetic Point Clouds and Analytic Grasp Metrics," *Proc. Robotics: Sci. Syst. (RSS)*, 2017.

[2] A. Murali, B. Sundaralingam, Y.-W. Chao, W. Yuan, J. Yamada, M. Carlson, F. Ramos, S. Birchfield, D. Fox, and C. Eppner, "GraspGen: A Diffusion-based Framework for 6-DOF Grasping with On-Generator Training," *arXiv:2507.13097*, 2025.

[3] J. Li and D. J. Cappelleri, "Sim-Grasp: Learning 6-DOF Grasp Policies for Cluttered Environments Using a Synthetic Benchmark," *arXiv:2405.00841*, 2024.

[4] B. Han, Y.-W. Chao, E. Coumans, C. Eppner, B. Sundaralingam, J. Deng, S. Birchfield, and A. Murali, "GraspGen-X: Cross-Embodiment 6-DOF Diffusion-based Grasping," *arXiv:2606.00998*, 2026.

[5] M. Torne, A. Mahajan, A. Bhat, and C. Finn, "Freeform Preference Learning for Robotic Manipulation," *arXiv:2606.32027*, 2026.

[6] T. Liu, R. Dazeley, B. Champion, and A. Cosgun, "Optimizing Robotic Placement via Grasp-Dependent Feasibility Prediction," *arXiv:2512.18922*, 2025.

[7] H. Wang, J. Bowden, C. Crosby, and S. Bansal, "Offline Policy Evaluation for Manipulation Policies via Discounted Liveness Formulation," *arXiv:2605.11479*, 2026.

[8] M. Yu, Y. Jiang, Y. Jia, R. Yi, and X. Li, "CoorGrasp: Coordinated Contact Control for Adaptive Dexterous Grasping Under Uncertainty," *arXiv:2607.03557*, 2026.

[9] S. Roth, "A Grammar of Machine Learning Workflows: Rejecting Data Leakage at Call Time," *arXiv:2603.10742*, 2026.

[10] E. C. Joyce, Q. Zhao, N. Burgdorfer, L. Wang, and P. Mordohai, "Consensus-Driven Uncertainty for Robotic Grasping based on RGB Perception," *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2025.

[11] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, "Implicit Behavioral Cloning," *Conf. Robot Learning (CoRL)*, 2021.

[12] G. Singh, S. Kalwar, M. F. Karim, B. Sen, N. Govindan, S. Sridhar, and K. M. Krishna, "Constrained 6-DoF Grasp Generation on Complex Shapes for Improved Dual-Arm Manipulation," *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2024.

[13] Anonymous, "When Geometry Is Not Enough: Object-Dependent Failure Modes of Learning-to-Rank Grasp Selection in Open-World Manipulation," *unpublished manuscript*, 2026.

TABLE III
FULL MECHANISTIC DETAIL FOR EVERY RULED-OUT METHOD.

#	Method	Mechanism	Result	Why it failed
1	OT-CFM candidate generation	Minibatch-OT-coupled conditional flow matching, replacing random CoM sampling	Pooled −10.0pp vs. baseline, p=0.0025; negative on all 7 objects	Minibatch OT coupling ignores conditioning (object identity) – training/inference distribution mismatch
2	C^2OT condition-aware OT fix	Condition-aware cost-matrix reweighting	Partial, inconsistent mitigation; hardest object showed no improvement	Fix validated only on generation-quality metrics, not physical execution
3	MPC-style world-model correction	Small-range action search before gripper close	Net negative, 3 pilots (−9.3/−18.7/−13.3pp)	Real-time in-loop correction search consistently made things worse
4	Consensus (median-pose) selection	Pick pool-median candidate, not lowest IK error	Effective on SO-ARM101 (Fisher's exact significant); no cross-embodiment replication under two noise models. Re-verified under section IV-H's fix (Pear 80%/70%, Mustard 80%/70%, Cracker 50%/50%, all null)	Training-free heuristic; genuine generalization failure, not a bug, and not an RNG-bug artifact
5	AR-mediated active calibration	Rollout prediction shown via AR; active human-judgment sampling	Active worse than passive (p=0.0048); refinement null (p=0.32–0.73)	No benefit from active solicitation over passive logging
6	Cross-embodiment reranking	Pooled-data reranker for live candidate selection	$p<0.0001\times5$ → null after 1st correction; 2nd leak found by auto-tagger → null confirmed, accuracy 0.8236→0.1327	No causally valid pre-execution signal predicts outcome; best valid feature: 0.06 correlation
7	Binary contact hard-stop	Stop descend on any detected contact	*Original* (RNG-bug-affected): worse, 40% vs. 70%. **Re-verified** (section IV-H's fix): null, 65% vs. 65%, p=1.0000, 2 discordant pairs	Original finding does not replicate under valid pairing – no measurable effect
8	Sustained-contact stop	Require 5 consecutive contact-positive steps	*Original*: promising batch 1 (75%/55%), opposite-direction batch 2 (65%/50%), combined p=1.0000. **Re-verified**: both batches agree, 75% vs. 70%, p=1.0000 each	Original instability was itself likely an RNG-bug artifact; valid pairing gives a stable, consistent null
9	Force-admittance damping	Progress damped by contact force magnitude	*Original*: worse in 2 calibrations (55%/75%, 50%/75%). **Re-verified**: 75% vs. 75%, *zero* discordant pairs across n=20	Original mechanistic story (exposure-window extension) does not hold under valid pairing – no measurable effect, not a harmful one
10	Gripper pre-narrowing	Narrow gripper before descend	*Original*: worse, 20% vs. 50%, p=0.11 (n.s.). **Re-verified**: worse, 15% vs. 75%, **p=0.0005**	The only one of the four with a genuine, replicated, strengthened effect – permanently reducing pre-descent clearance measurably hurts
11	World-model sim-to-real transfer	Train in sim, deploy with little/no real data	Not piloted – closed at feasibility stage	Working prior art needs infrastructure (thousands of real demos, or video-diffusion-scale backbone) this project lacks
12	Energy-based candidate scoring (EBM v1→v2)	Implicit scalar compatibility model, motivated by Implicit Behavioral Cloning [11] and its grasp-specific descendant [12]; v1: static BCE + CEM sampling; v2 adds hard negatives mined via a 3-iteration CEM search using the model's own current weights	v1: catastrophic, 0% on 2 of 3 objects (p<0.001 both). v2: recovers to 80%/100%/26% (Pear/TomatoSoupCan/CrackerBox), pooled −3.7pp vs. baseline (n.s., p=0.29), significant win on TomatoSoupCan (p=0.041)	v1's static negatives left an under-constrained pose region the inference-time CEM search could exploit (found ∼1.7 SD from the training mean, scored 99% confident despite no training support there); v2's mined negatives close that gap, reaching parity with baseline but not clearly exceeding it